

# Magnus v1.1

Modelo de produção honesto: mesma arquitetura do v1 (YOLO11s, 9,4M parâmetros), treinada apenas nas 6 classes saudáveis após o diagnóstico cirúrgico da matriz de confusão. mAP50 sobe de 0,655 para 0,770 (+11,5 pp) sem aumento de modelo nem de dados — puramente removendo classes incoerentes ou famintas de dados. Pronto para deploy.

## 1. Resumo executivo

O Magnus v1.1 é a primeira versão do projeto destinada a uso real. Toma a decisão deliberada de detectar **apenas o que consegue detectar bem**: descarta as classes *risco* (taxonomia incoerente, fundia scratch+Crack+Paint-Crack) e *oxidacao* (apenas 17 instâncias no test split — dado insuficiente). O resultado é um sistema que **promete 6 e entrega 6**, com latência baixa o suficiente para inspeção em tempo quase real e custo zero de infraestrutura.

### Status

| Componente                      | Status                                        |
|---------------------------------|-----------------------------------------------|
| Modelo treinado                 | best.pt no HF Hub (Guguinhaxd/magnus-v1-prod) |
| Backend                         | FastAPI em produção (Hugging Face Spaces)     |
| Frontend                        | Magnus UI em produção (Vercel)                |
| Banco de dados                  | Supabase Postgres + Storage                   |
| Classes detectadas              | 6 (todas com mAP50 ≥ 0,58)                    |
| Latência inferência (L4)        | 2,8 ms                                        |
| Latência inferência (CPU, prod) | ~80 ms estimado                               |
| Custo mensal                    | R\$ 0,00 (free tier em todos os serviços)     |

## 2. Trajetória v0 → v1 → v1.1

| Aspecto         | v0                                                                                                                       | v1                | v1.1               |
|-----------------|--------------------------------------------------------------------------------------------------------------------------|-------------------|--------------------|
| Arquitetura     | YOLOv8n                                                                                                                  | YOLO11s           | YOLO11s            |
| Parâmetros      | 3,0 M                                                                                                                    | 9,4 M             | 9,4 M              |
| Classes         | 4                                                                                                                        | 8                 | 6                  |
| Fontes dataset  | 1                                                                                                                        | 5 curadas (de 13) | 5 curadas (subset) |
| GPU treino      | T4 free                                                                                                                  | T4 free           | L4 (Colab Pro+)    |
| Tempo treino    | ~42 min                                                                                                                  | ~4 h              | 1,3 h              |
| Epochs efetivos | 80                                                                                                                       | 150 (platô ~60)   | 120 (platô ~75)    |
| mAP@50 agregado | 0,989                                                                                                                    | 0,655             | 0,770              |
| mAP@50–95       | 0,864                                                                                                                    | 0,428             | 0,517              |
| Lição central   | viabilidade do pipeline cloud-first. Não importa mais que arquitetura, a decisão honesta de classes incoerentes recupera |                   |                    |

### Por que esta versão é qualitativamente diferente

v0 e v1 eram experimentos. v0 provou que o pipeline funciona ponta a ponta; v1 expandiu para diagnosticar limites. **v1.1 é a primeira versão pensada para produção**: cada classe que ele expõe foi validada empiricamente como confiável. Em ambiente industrial, um detector que alucina defeitos onde não há destrói a confiança do operador mais rapidamente do que um detector com cobertura menor — e por isso a decisão informada-por-evidência de cortar duas classes é mais valiosa que o ganho bruto de mAP.

### 3. Setup do treino

| Atributo          | Valor                                                                                        |
|-------------------|----------------------------------------------------------------------------------------------|
| Hardware          | NVIDIA L4 (Colab Pro+), 24 GB VRAM                                                           |
| Epochs            | 120 (early stopping com patience=40, sem disparo)                                            |
| Batch / imgsz     | 32 / 640 (vs 16 no v1 — VRAM da L4 permite)                                                  |
| Otimizador        | AdamW automático, lr=0,001, cosine LR schedule                                               |
| Augmentation      | mosaic 1.0 (close em 105), mixup 0,15, copy_paste 0,3, erasing 0,4, degrees ±10°, fliplr 0,5 |
| Dataset splits    | 3.208 train (214 bg) · 531 val (49 bg) · 354 test (24 bg)                                    |
| Classes (6)       | casca_de_laranja, escorrimento, bolha, water_spotting, descascamento, sujeira                |
| Removidas (vs v1) | risco (taxonomia incoerente), oxidacao (17 instâncias no test, fome de dado)                 |
| Dedup             | por imagem-fonte, escopo de workspace                                                        |
| Avaliação         | test split com TTA (test-time augmentation)                                                  |

### 4. Métricas v1.1 (test split, TTA)

#### Globais

| Métrica   | v0    | v1    | v1.1  | Δ vs v1 |
|-----------|-------|-------|-------|---------|
| mAP@50    | 0,989 | 0,655 | 0,770 | +0,115  |
| mAP@50–95 | 0,864 | 0,428 | 0,517 | +0,089  |
| Precision | 0,970 | 0,807 | 0,817 | +0,010  |
| Recall    | 0,966 | 0,617 | 0,734 | +0,117  |

#### Por classe

| Classe           | v1 mAP50 | v1.1 mAP50 | Δ      | Veredito |
|------------------|----------|------------|--------|----------|
| casca_de_laranja | 0,951    | 0,978      | +0,027 | produção |
| water_spotting   | 0,928    | 0,843      | −0,085 | produção |
| escorrimento     | 0,849    | 0,782      | −0,067 | produção |
| bolha            | 0,836    | 0,776      | −0,060 | produção |
| sujeira          | 0,732    | 0,657      | −0,075 | produção |
| descascamento    | 0,493    | 0,585      | +0,092 | produção |

|          |       |   |          |                        |
|----------|-------|---|----------|------------------------|
| risco    | 0,253 | — | removida | incoerência taxonômica |
| oxidacao | 0,198 | — | removida | fome de dados          |

**Leitura honesta das pequenas quedas** em water\_spotting, escorrimento, bolha e sujeira: são em parte variação de split (o dataset mudou ao remover imagens cujo único defeito era risco ou oxidacao) e em parte trade-off natural — o modelo antes alocava capacidade para 8 classes, agora alocaria para 6, mas ficou com mais sinal por classe. O agregado é o que conta, e descascamento subiu 9 pp como prova de que risco e descascamento estavam competindo internamente.

## 5. Análise da matriz de confusão

Diagonal forte em todas as 6 classes, sem confusão entre classes — apenas trocas com background (esperado em defeitos sutis em texturas variadas):

| Classe           | Acertos | Confunde com...                           | Diagnóstico                            |
|------------------|---------|-------------------------------------------|----------------------------------------|
| casca_de_laranja | 74      | nada (apenas background)                  | limpo, mAP 0,978                       |
| water_spotting   | 96      | nada                                      | diagonal forte                         |
| bolha            | 83      | 1 casca, 1 escorrimento, 3 water_spotting | distúrbio mínimo entre classes         |
| escorrimento     | 50      | nada                                      | limpo                                  |
| sujeira          | 37      | nada (37 ↔ background)                    | trade-off precisão/cobertura natural   |
| descascamento    | 22      | nada                                      | 9 pp acima do v1, sem risco competindo |

### Por que risco causava tanto estrago no v1

No v1, a classe risco aparecia em **54 backgrounds** (predição falsa: o modelo via risco onde não havia), **40 riscos reais** eram preditos como background (não detectava), e havia confusão visível com escorrimento. O motivo: tínhamos fundido scratch + Crack + Paint-Crack em uma classe — arranhão fino, rachadura visível e linha pintada são visualmente distintos. O modelo era forçado a aprender um conceito sem identidade visual coerente. Removendo essa classe ruidosa, todas as outras herdaram a capacidade da rede que estava sendo desperdiçada.

## 6. Deploy em produção

Para ativar o v1.1 no Hugging Face Space, basta atualizar a variável de ambiente:

```
BAJA_MODEL_WEIGHTS = hf://Guguinhaxd/magnus-v1-prod/best.pt
```

O backend baixa o checkpoint do Hub no boot via huggingface\_hub, cacheia, e carrega no Ultralytics. *Hot-swap* sem rebuild do container.

### Frontend já está atualizado

Os mapas de severidade e os rótulos amigáveis em ResultsPanel.tsx e DetectionOverlay.tsx incluem agora *descascamento* (severidade alta — expõe substrato) e *sujeira* (severidade baixa — contaminação superficial). Vercel redeploys automaticamente a cada push.

## 7. Lição central e próximos passos

### A frase que resume o ciclo

*“Entre v0 e v1.1, o ganho não veio de mais parâmetros nem de GPU melhor — veio de entender os dados. Curadoria e taxonomia honesta valeram mais que 64 mil imagens extras.”*

### v2 — próximo ciclo (dataset próprio)

| Ação                                                                                                               | Resolve                                             |
|--------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| Fotografar chassi BAJA real (300–500 imgs, iluminação controlada, câmera USB 4K) vs. setup industrial → campo real | do US8                                              |
| Rotular internamente com taxonomia própria — sem fusões de defeitos visuais                                        | de 10 para 6 classes distintas como classe coerente |

Coleta dirigida de oxidação via active learning (rodar v1.1 nos 15k weld resins do BAJA com ocaor e ocaor de ocaor)

Separar bolha de solvent\_pop se amostras permitirem

granularidade técnica para inspeção real

## 8. Artefatos do ciclo

| Artefato                  | Localização                                                      |
|---------------------------|------------------------------------------------------------------|
| Checkpoint v1.1 (19,2 MB) | hf://Guguinhaxd/magnus-v1-prod/best.pt                           |
| Pipeline dataset (subset) | training/v2/build_dataset.py (parâmetro classes=...)             |
| Receita de treino         | training/v2/train.py                                             |
| Avaliação TTA + matriz    | training/v2/evaluate.py                                          |
| Frontend atualizado       | frontend/app/components/{ResultsPanel,DetectionOverlay}.tsx      |
| Snapshots anteriores      | docs/progression/magnus_v0_snapshot.pdf · magnus_v1_snapshot.pdf |

Gerado em 2026-06-12 20:38 · Documento auto-suficiente para o projeto de tracking de progressão. Versão de produção; sucessora prevista: magnus\_v2 (dataset próprio de chassi BAJA, reintroduz risco e oxidacao com base sólida).